EXPERIMENT-04

Backpropagation Neural Network: Dataset-Based Python Programs

Name: B.ANJALI Reg.No: 192371062

4. Build an Artificial Neural Network by implementing the Backpropagation algorithm and test the same using appropriate data sets.

Problem 1: XOR Gate

Aim

To build an Artificial Neural Network using the Backpropagation algorithm that learns the XOR function stored in a CSV file.

Procedure

1. Save the training examples in xor_data.csv with numeric input attributes and the 0/1 target in the last column.
2. Run the Python program from the same folder as the CSV file.
3. Read the data, normalize the inputs, and initialize the weights randomly and the biases to zero.
4. Forward pass: compute the hidden layer and the output using the sigmoid function.
5. Backward pass: compute the output and hidden errors, then update the weights and biases (learning rate 0.5); repeat for 10000 epochs.
6. Print the error during training, then compare the predicted outputs with the actual outputs and display the accuracy.

Dataset File / Input

File name: xor_data.csv

The CSV file contains a header row followed by these training examples:

Input1	Input2	Output
0	0	0
0	1	1
1	0	1
1	1	0

Code

```
import numpy as np
import pandas as pd

df = pd.read_csv("xor_data.csv")
X = df.iloc[:, :-1].values.astype(float)
y = df.iloc[:, -1].values.reshape(-1, 1)
X = X / X.max(axis=0)                              # normalize inputs

np.random.seed(1)
W1 = np.random.uniform(-1, 1, (X.shape[1], 4))     # input -> hidden weights
b1 = np.zeros((1, 4))
W2 = np.random.uniform(-1, 1, (4, 1))              # hidden -> output weights
b2 = np.zeros((1, 1))
sigmoid = lambda z: 1 / (1 + np.exp(-z))
lr = 0.5

for epoch in range(10001):
    h = sigmoid(X @ W1 + b1)                       # forward pass
    out = sigmoid(h @ W2 + b2)
    d2 = (y - out) * out * (1 - out)               # backward pass
    d1 = (d2 @ W2.T) * h * (1 - h)
    W2 += lr * h.T @ d2 / len(X)
```

```
    b2 += lr * d2.mean(axis=0)
    W1 += lr * X.T @ d1 / len(X)
    b1 += lr * d1.mean(axis=0)
    if epoch % 2000 == 0:
        print("Epoch", epoch, "Error:", round(float(np.mean((y - out) ** 2)), 4))

print("Predicted:", out.round(3).ravel())
print("Actual   :", y.ravel())
print("Accuracy :", (out.round() == y).mean() * 100, "%")
```

Output

```
Epoch 0 Error: 0.2497
Epoch 2000 Error: 0.1562
Epoch 4000 Error: 0.0172
Epoch 6000 Error: 0.0051
Epoch 8000 Error: 0.0028
Epoch 10000 Error: 0.0018
Predicted: [0.035 0.952 0.968 0.053]
Actual   : [0 1 1 0]
Accuracy : 100.0 %
```

Result

The network (2 inputs, 4 hidden neurons, 1 output) learns the non-linear XOR function. The error falls from 0.2497 to 0.0018 over 10000 epochs, and the predictions (0.035, 0.952, 0.968, 0.053) round to the actual outputs 0, 1, 1, 0, giving 100% accuracy.

Problem 2: Student Pass Prediction

Aim

To train a backpropagation neural network to predict whether a student passes from study hours and sleep hours stored in a CSV file.

Procedure

1. Save the training examples in studyhours_data.csv with numeric input attributes and the 0/1 target in the last column.
2. Run the Python program from the same folder as the CSV file.
3. Read the data, normalize the inputs, and initialize the weights randomly and the biases to zero.
4. Forward pass: compute the hidden layer and the output using the sigmoid function.
5. Backward pass: compute the output and hidden errors, then update the weights and biases (learning rate 0.5); repeat for 10000 epochs.
6. Print the error during training, then compare the predicted outputs with the actual outputs and display the accuracy.

Dataset File / Input

File name: studyhours_data.csv

StudyHours	SleepHours	Pass
1	7	0
2	6	0
3	8	0
4	5	0
5	7	1
6	6	1
7	8	1
8	5	1

9	7	1
3	4	0
6	8	1
8	6	1

The CSV file contains a header row followed by these training examples:

Pass: 1 = Pass, 0 = Fail.

Code

```
import numpy as np
import pandas as pd

df = pd.read_csv("studyhours_data.csv")
X = df.iloc[:, :-1].values.astype(float)
y = df.iloc[:, -1].values.reshape(-1, 1)
X = X / X.max(axis=0)                              # normalize inputs

np.random.seed(1)
W1 = np.random.uniform(-1, 1, (X.shape[1], 4))     # input -> hidden weights
b1 = np.zeros((1, 4))
W2 = np.random.uniform(-1, 1, (4, 1))              # hidden -> output weights
b2 = np.zeros((1, 1))
sigmoid = lambda z: 1 / (1 + np.exp(-z))
lr = 0.5

for epoch in range(10001):
    h = sigmoid(X @ W1 + b1)                       # forward pass
    out = sigmoid(h @ W2 + b2)
    d2 = (y - out) * out * (1 - out)               # backward pass
    d1 = (d2 @ W2.T) * h * (1 - h)
    W2 += lr * h.T @ d2 / len(X)
    b2 += lr * d2.mean(axis=0)
    W1 += lr * X.T @ d1 / len(X)
    b1 += lr * d1.mean(axis=0)
    if epoch % 2000 == 0:
        print("Epoch", epoch, "Error:", round(float(np.mean((y - out) ** 2)), 4))

print("Predicted:", out.round(3).ravel())
print("Actual   :", y.ravel())
print("Accuracy :", (out.round() == y).mean() * 100, "%")
```

Output

```
Epoch 0 Error: 0.2468
Epoch 2000 Error: 0.0242
Epoch 4000 Error: 0.0093
Epoch 6000 Error: 0.0052
Epoch 8000 Error: 0.0034
Epoch 10000 Error: 0.0024
Predicted: [0.001 0.002 0.066 0.098 0.882 0.973 0.997 0.998 0.999 0.005 0.99  0.998]
Actual   : [0 0 0 0 1 1 1 1 1 0 1 1]
Accuracy : 100.0 %
```

Result

The error falls from 0.2468 to 0.0024 during training. All 12 predictions round to the actual class, so the accuracy is 100%.

Problem 3: Diabetes Prediction

Aim

To train a backpropagation neural network to predict diabetes from glucose level and BMI stored in a CSV file.

Procedure

1. Save the training examples in glucose_data.csv with numeric input attributes and the 0/1 target in the last column.
2. Run the Python program from the same folder as the CSV file.
3. Read the data, normalize the inputs, and initialize the weights randomly and the biases to zero.
4. Forward pass: compute the hidden layer and the output using the sigmoid function.
5. Backward pass: compute the output and hidden errors, then update the weights and biases (learning rate 0.5); repeat for 10000 epochs.
6. Print the error during training, then compare the predicted outputs with the actual outputs and display the accuracy.

Dataset File / Input

File name: glucose_data.csv

The CSV file contains a header row followed by these training examples:

Glucose	BMI	Diabetic
85	22.5	0
92	24.1	0
110	26.0	0
100	23.4	0
120	27.5	0
130	31.2	1
150	33.5	1
145	29.8	1
165	35.2	1
140	32.0	1
105	25.3	0
155	30.1	1
90	28.0	0
175	36.8	1

Diabetic: 1 = Yes, 0 = No.

Code

```
import numpy as np
import pandas as pd

df = pd.read_csv("glucose_data.csv")
X = df.iloc[:, :-1].values.astype(float)
y = df.iloc[:, -1].values.reshape(-1, 1)
X = X / X.max(axis=0)                              # normalize inputs

np.random.seed(1)
W1 = np.random.uniform(-1, 1, (X.shape[1], 4))     # input -> hidden weights
b1 = np.zeros((1, 4))
W2 = np.random.uniform(-1, 1, (4, 1))              # hidden -> output weights
b2 = np.zeros((1, 1))
sigmoid = lambda z: 1 / (1 + np.exp(-z))
lr = 0.5

for epoch in range(10001):
    h = sigmoid(X @ W1 + b1)                       # forward pass
    out = sigmoid(h @ W2 + b2)
    d2 = (y - out) * out * (1 - out)               # backward pass
    d1 = (d2 @ W2.T) * h * (1 - h)
    W2 += lr * h.T @ d2 / len(X)
    b2 += lr * d2.mean(axis=0)
    W1 += lr * X.T @ d1 / len(X)
    b1 += lr * d1.mean(axis=0)
```

```
    if epoch % 2000 == 0:
        print("Epoch", epoch, "Error:", round(float(np.mean((y - out) ** 2)), 4))

print("Predicted:", out.round(3).ravel())
print("Actual   :", y.ravel())
print("Accuracy :", (out.round() == y).mean() * 100, "%")
```

Output

```
Epoch 0 Error: 0.2497
Epoch 2000 Error: 0.053
Epoch 4000 Error: 0.0195
Epoch 6000 Error: 0.012
Epoch 8000 Error: 0.0084
Epoch 10000 Error: 0.0063
Predicted: [0.001 0.002 0.036 0.005 0.206 0.807 0.987 0.938 0.997 0.947 0.015 0.979
 0.007 0.999]
Actual   : [0 0 0 0 0 1 1 1 1 1 0 1 0 1]
Accuracy : 100.0 %
```

Result

The error falls from 0.2497 to 0.0063. All 14 predictions round to the actual class, so the accuracy is 100%; the least confident prediction is 0.206 for a sample whose actual class is 0.

Problem 4: Loan Approval

Aim

To train a backpropagation neural network to predict loan approval from income and credit score stored in a CSV file.

Procedure

1. Save the training examples in loan_data.csv with numeric input attributes and the 0/1 target in the last column.
2. Run the Python program from the same folder as the CSV file.
3. Read the data, normalize the inputs, and initialize the weights randomly and the biases to zero.
4. Forward pass: compute the hidden layer and the output using the sigmoid function.
5. Backward pass: compute the output and hidden errors, then update the weights and biases (learning rate 0.5); repeat for 10000 epochs.
6. Print the error during training, then compare the predicted outputs with the actual outputs and display the accuracy.

Dataset File / Input

File name: loan_data.csv

The CSV file contains a header row followed by these training examples:

Income	CreditScore	Approved
25	550	0
30	600	0
45	650	1
60	700	1
80	750	1
35	580	0
55	680	1
70	720	1
28	520	0
90	780	1
40	630	0
65	710	1

Approved: 1 = Yes, 0 = No.

Code

```
import numpy as np
import pandas as pd

df = pd.read_csv("loan_data.csv")
X = df.iloc[:, :-1].values.astype(float)
y = df.iloc[:, -1].values.reshape(-1, 1)
X = X / X.max(axis=0)                              # normalize inputs

np.random.seed(1)
W1 = np.random.uniform(-1, 1, (X.shape[1], 4))     # input -> hidden weights
b1 = np.zeros((1, 4))
W2 = np.random.uniform(-1, 1, (4, 1))              # hidden -> output weights
b2 = np.zeros((1, 1))
sigmoid = lambda z: 1 / (1 + np.exp(-z))
lr = 0.5

for epoch in range(10001):
    h = sigmoid(X @ W1 + b1)                       # forward pass
    out = sigmoid(h @ W2 + b2)
    d2 = (y - out) * out * (1 - out)               # backward pass
    d1 = (d2 @ W2.T) * h * (1 - h)
    W2 += lr * h.T @ d2 / len(X)
    b2 += lr * d2.mean(axis=0)
    W1 += lr * X.T @ d1 / len(X)
    b1 += lr * d1.mean(axis=0)
    if epoch % 2000 == 0:
        print("Epoch", epoch, "Error:", round(float(np.mean((y - out) ** 2)), 4))

print("Predicted:", out.round(3).ravel())
print("Actual   :", y.ravel())
print("Accuracy :", (out.round() == y).mean() * 100, "%")
```

Output

```
Epoch 0 Error: 0.2469
Epoch 2000 Error: 0.0443
Epoch 4000 Error: 0.0262
Epoch 6000 Error: 0.0202
Epoch 8000 Error: 0.0165
Epoch 10000 Error: 0.0137
Predicted: [0.002 0.011 0.711 0.994 1.    0.048 0.982 0.999 0.004 1.    0.28  0.998]
Actual   : [0 0 1 1 1 0 1 1 0 1 0 1]
Accuracy : 100.0 %
```

Result

The error falls from 0.2469 to 0.0137. All 12 predictions round to the actual class, so the accuracy is 100%; the borderline applicants (0.711 and 0.28) are the least confident predictions.

Problem 5: Flower Classification

Aim

To train a backpropagation neural network to classify flowers as Setosa or Versicolor from petal measurements stored in a CSV file.

Procedure

1. Save the training examples in flower_data.csv with numeric input attributes and the 0/1 target in the last column.
2. Run the Python program from the same folder as the CSV file.
3. Read the data, normalize the inputs, and initialize the weights randomly and the biases to zero.
4. Forward pass: compute the hidden layer and the output using the sigmoid function.
5. Backward pass: compute the output and hidden errors, then update the weights and biases (learning rate 0.5); repeat for 10000 epochs.
6. Print the error during training, then compare the predicted outputs with the actual outputs and display the accuracy.

Dataset File / Input

File name: flower_data.csv

The CSV file contains a header row followed by these training examples:

PetalLength	PetalWidth	Species
1.4	0.2	0
1.3	0.2	0
1.5	0.2	0
1.7	0.4	0
1.4	0.3	0
4.7	1.4	1
4.5	1.5	1
4.9	1.5	1
4.0	1.3	1
4.6	1.5	1
3.9	1.1	1
1.6	0.2	0

Species: 0 = Setosa, 1 = Versicolor.

Code

```
import numpy as np
import pandas as pd

df = pd.read_csv("flower_data.csv")
X = df.iloc[:, :-1].values.astype(float)
y = df.iloc[:, -1].values.reshape(-1, 1)
X = X / X.max(axis=0)                              # normalize inputs

np.random.seed(1)
W1 = np.random.uniform(-1, 1, (X.shape[1], 4))     # input -> hidden weights
b1 = np.zeros((1, 4))
W2 = np.random.uniform(-1, 1, (4, 1))              # hidden -> output weights
b2 = np.zeros((1, 1))
sigmoid = lambda z: 1 / (1 + np.exp(-z))
lr = 0.5

for epoch in range(10001):
    h = sigmoid(X @ W1 + b1)                       # forward pass
    out = sigmoid(h @ W2 + b2)
    d2 = (y - out) * out * (1 - out)               # backward pass
    d1 = (d2 @ W2.T) * h * (1 - h)
    W2 += lr * h.T @ d2 / len(X)
    b2 += lr * d2.mean(axis=0)
    W1 += lr * X.T @ d1 / len(X)
    b1 += lr * d1.mean(axis=0)
    if epoch % 2000 == 0:
        print("Epoch", epoch, "Error:", round(float(np.mean((y - out) ** 2)), 4))

print("Predicted:", out.round(3).ravel())
print("Actual   :", y.ravel())
print("Accuracy :", (out.round() == y).mean() * 100, "%")
```

Output

```
Epoch 0 Error: 0.2481
Epoch 2000 Error: 0.0034
Epoch 4000 Error: 0.0013
Epoch 6000 Error: 0.0008
Epoch 8000 Error: 0.0005
Epoch 10000 Error: 0.0004
Predicted: [0.011 0.01  0.012 0.037 0.017 0.986 0.988 0.989 0.978 0.988 0.959 0.013]
Actual   : [0 0 0 0 0 1 1 1 1 1 1 0]
Accuracy : 100.0 %
```

Result

The error falls from 0.2481 to 0.0004, the lowest of the five problems because the two classes are well separated. All 12 predictions round to the actual class, so the accuracy is 100%.